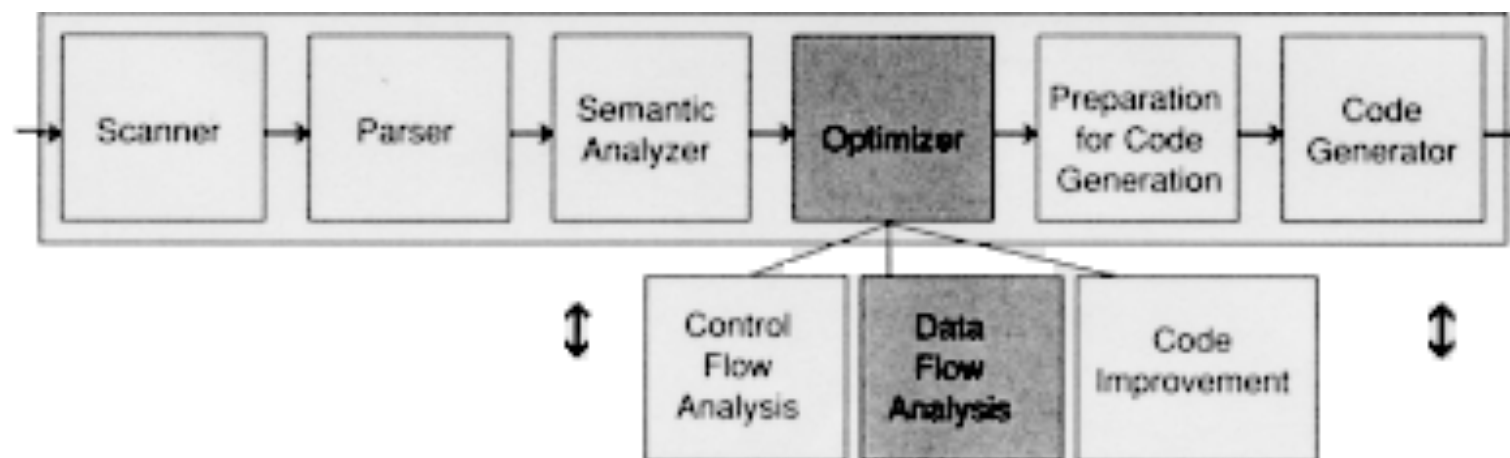


Dataflow Analyses

Code Optimization in Compilers



Correctness Above All!

It may seem obvious, but it bears repeating that optimization should not change the correctness of the generated code. Transforming the code to something that runs faster but incorrectly is of little value. It is expected that the unoptimized and optimized variants give the same output for all inputs. This may not hold for an incorrectly written program (e.g., one that uses an uninitialized variable).

What is Dataflow Analysis: general idea

A static analysis technique to track how information flows through a program

- Operates on a graph representation:
 - Typically the Control Flow Graph (CFG)
- Compute program properties (e.g., live variables, constant propagation, reaching definitions)
- Associates information to:
 - nodes (program points) or
 - edges
- Have two main directions:
 - Forward analysis (information flows along execution)
 - Backward analysis (information flows against execution)

Different analyses share the same underlying computational framework

Common Structure of Dataflow Analyses

Flow equations

- For each node:
 - $IN[n]$ = merge of predecessor (or successor) information
 - $OUT[n]$ = transfer function applied to $IN[n]$

Transfer function

- Models the effect of each statement
- Often expressed via gen/kill

Merge operator

- Combines information from multiple paths
- Common choices:
 - $\cup$ (union) $\rightarrow$ may analyses
 - $\cap$ (intersection) $\rightarrow$ must analyses

Computation

- Iterative solution until a fixed point

May vs Must Analyses

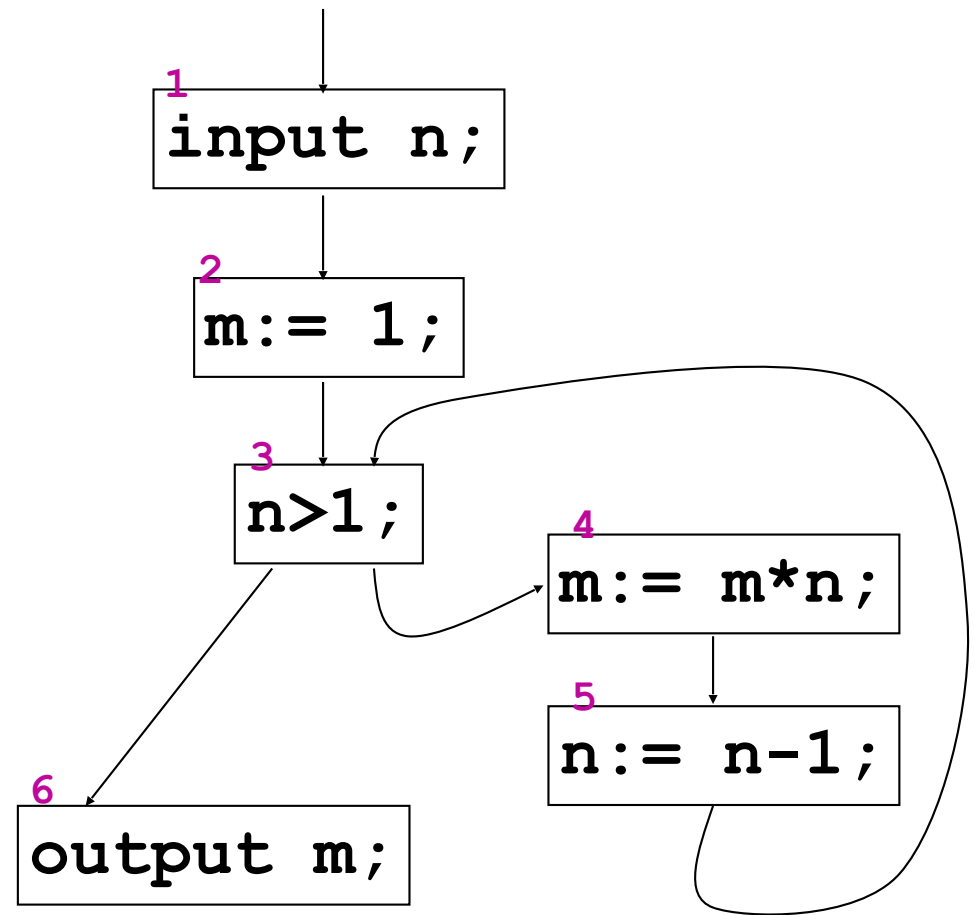
- Two types of dataflow analyses:
 - May (**possible**) analysis
 - Property may hold on some execution paths
 - Examples: reaching definitions, live variables
 - Must (**definitive**) analysis
 - Property holds on all execution paths
 - Examples: available expressions, definite assignment

Control flow graph

- Program commands are encoded by nodes in a control flow graph
- If a command S **may be directly followed** by a command T then the control flow graph must include a direct arc from the node encoding S to the node encoding T

Example

```
[ input n; ]1  
[ m := 1; ]2  
  [ while n>1 do ]3  
    [ m := m * n; ]4  
    [ n := n - 1; ]5  
  [ output m; ]6
```



Data-Flow analyses

We will see several data-flow analyses:

- **Liveness** analysis
- **Reaching definitions** analysis
- **Available expressions** analysis
- **Very busy expressions** analysis

all of them will differ either for **direction flow** or for being **definitive or possible**

Liveness Analysis

Liveness or Live Variables Analysis

- We need to translate the source program in the intermediate representation IR that uses a **large** (potentially unbounded) **number of registers**.
- but the program will be executed by a processor with a (finite and) **small number of registers**
- Two variables **a** and **b** can be stored in the same register when it turns out that **a** and **b** are **never simultaneously “used”**

IR: Three Address Code

Three-address instruction has at most three operands and is typically a combination of an assignment and a binary operator.

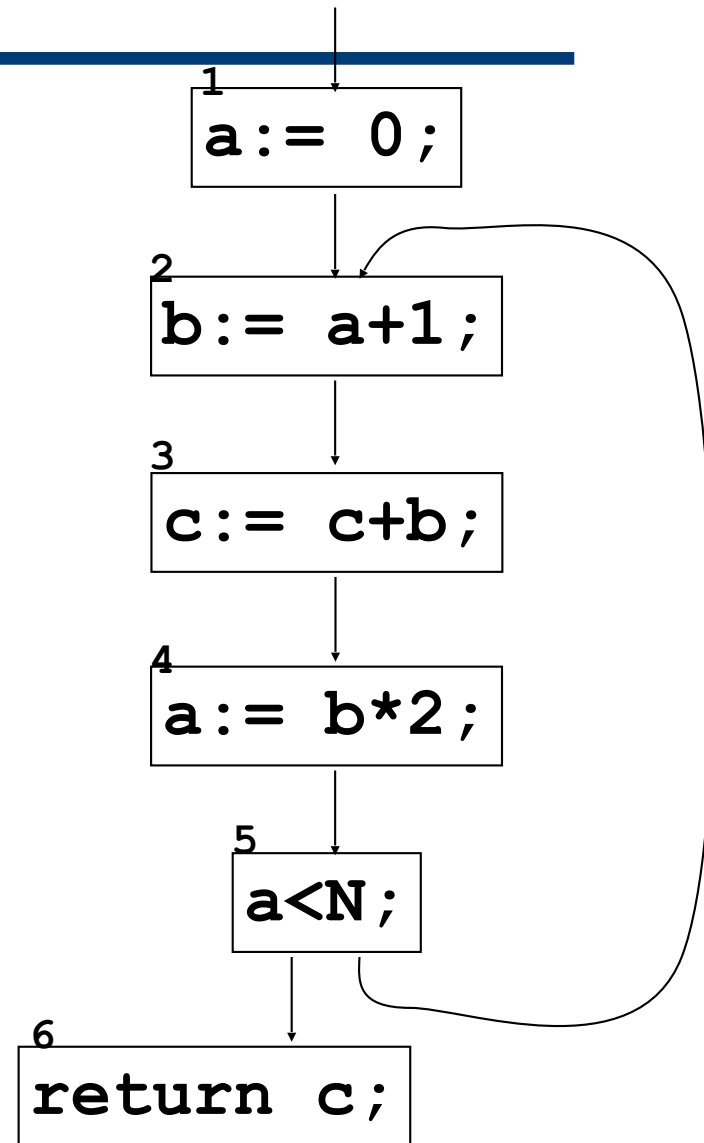
For example: $t1 := t2 + t3$.

The name derives from the use of three operands in these statements even though instructions with fewer operands may occur.

Example

```
a = 0;  
do {  
    b = a+1;  
    c += b;  
    a = b*2;  
}  
while (a<N);  
return c;
```

We want to know if **a** and **b** are simultaneously used.

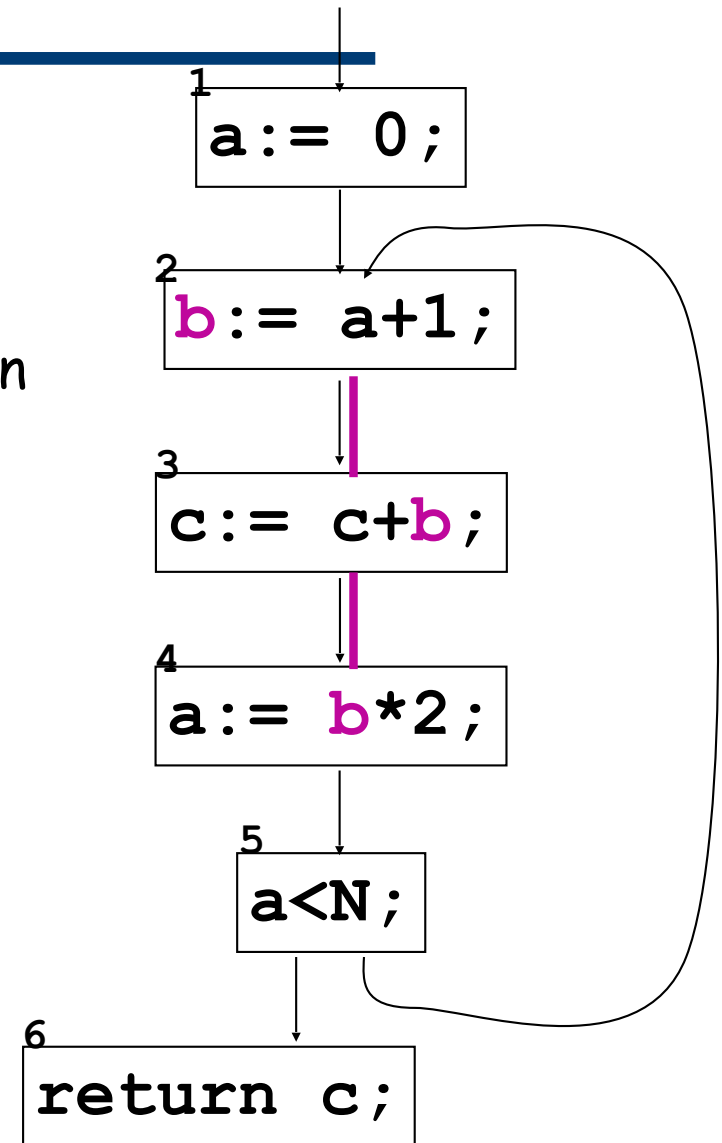


Live Variables Analysis

- A compiler needs to analyze programs in IR in order to find out which variables are simultaneously used
- A variable X is **live** at the exit of a command C if X stores a value which will be **actually used** in the future, that is, X will be used as R-value with no previous use as L-value
- A variable X which is not live at the exit of C is also called **dead** (this information can be used for dead code elimination)
- This is an undecidable property

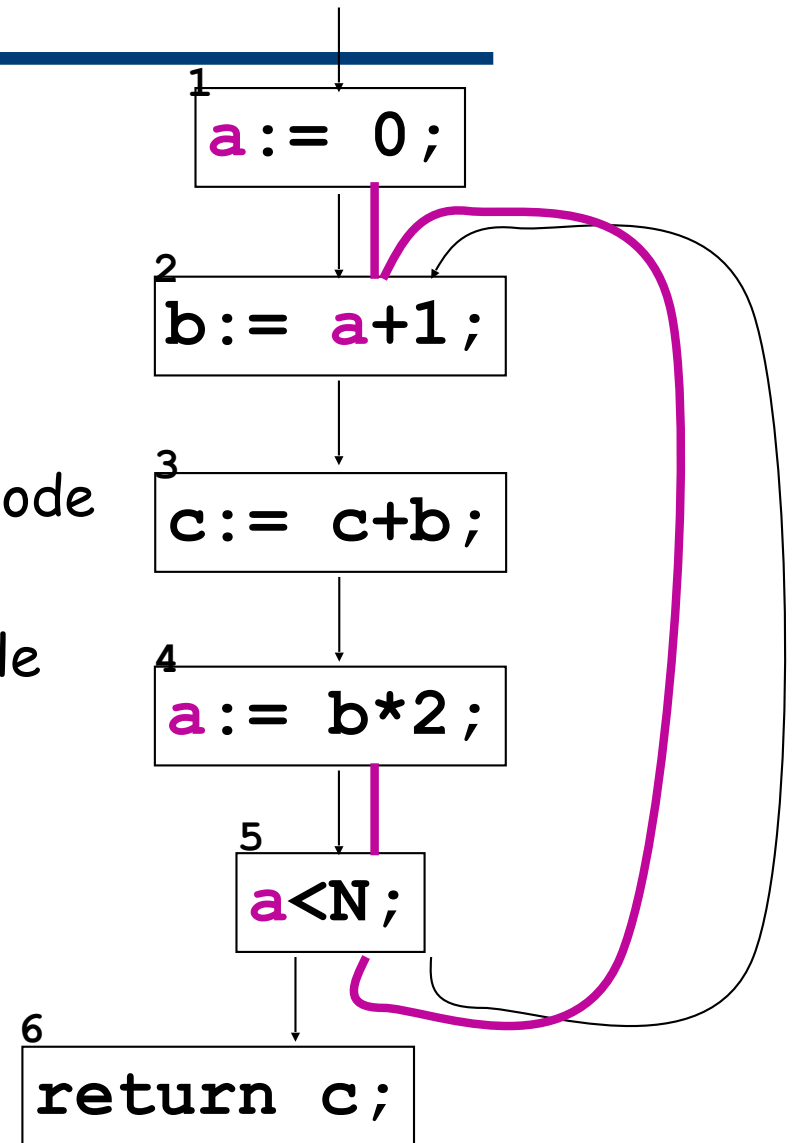
Back to the example

- A variable X is live when it stores a value which will be later used with no prior assignment to X
- The "last" use of the variable b as r-value is in command 4
- The variable b is used in command 4: it is therefore live along the arc $3 \rightarrow 4$
- Command 3 does not assign b , hence b is live along $2 \rightarrow 3$
- Command 2 assigns b . This means that the value of b along $1 \rightarrow 2$ will not be used later
- Thus, the "live range" of b turns out to be: $\{2 \rightarrow 3, 3 \rightarrow 4\}$



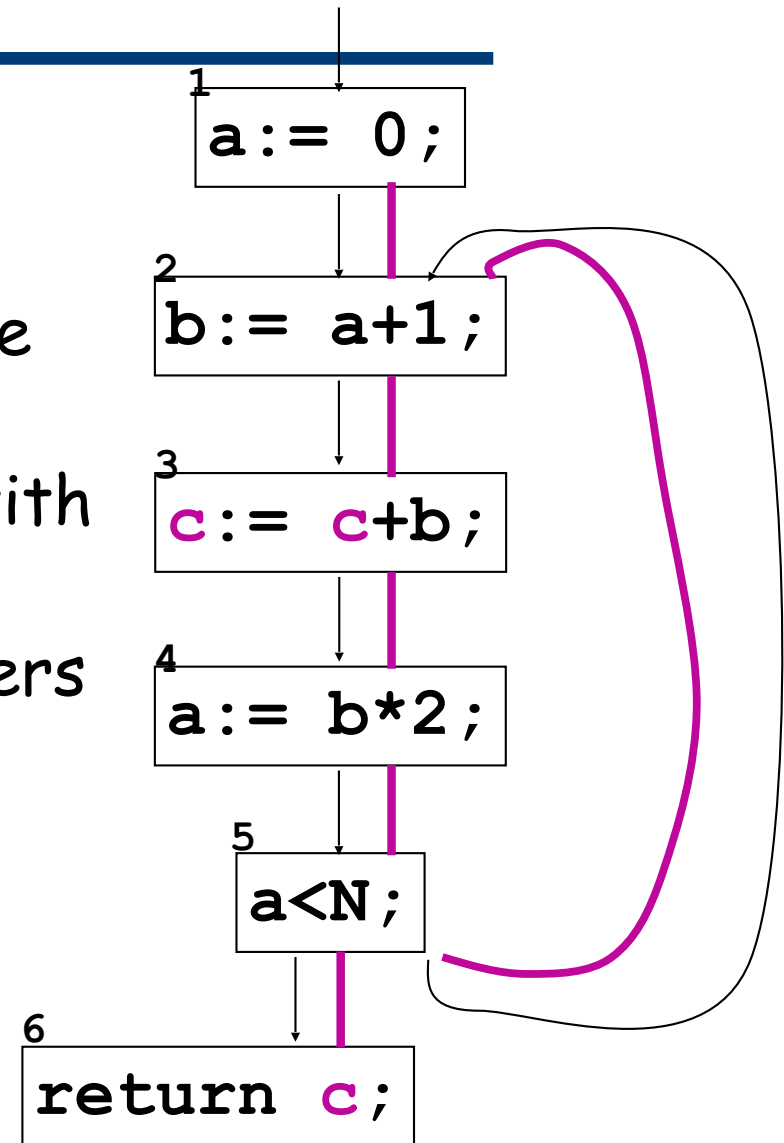
Live variables

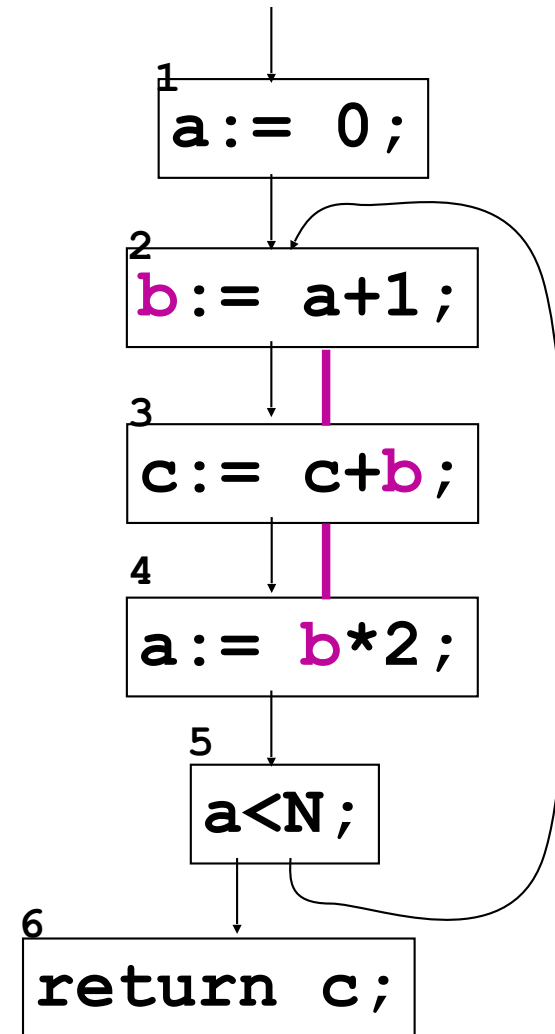
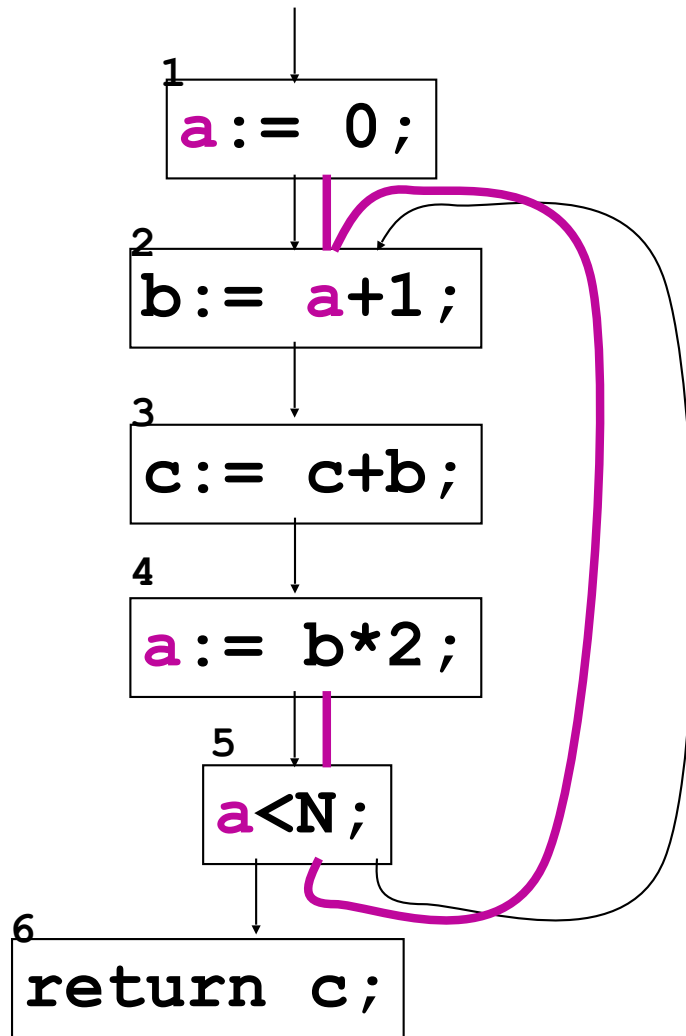
- **a** is live along $4 \rightarrow 5$ and $5 \rightarrow 2$
- **a** is live along $1 \rightarrow 2$
- **a** is not live along $2 \rightarrow 3$ and $3 \rightarrow 4$
- Even if the variable **a** stores a value in node 3, this value will not be later used, since node 4 assigns a new value to the variable **a**.



More on live variables

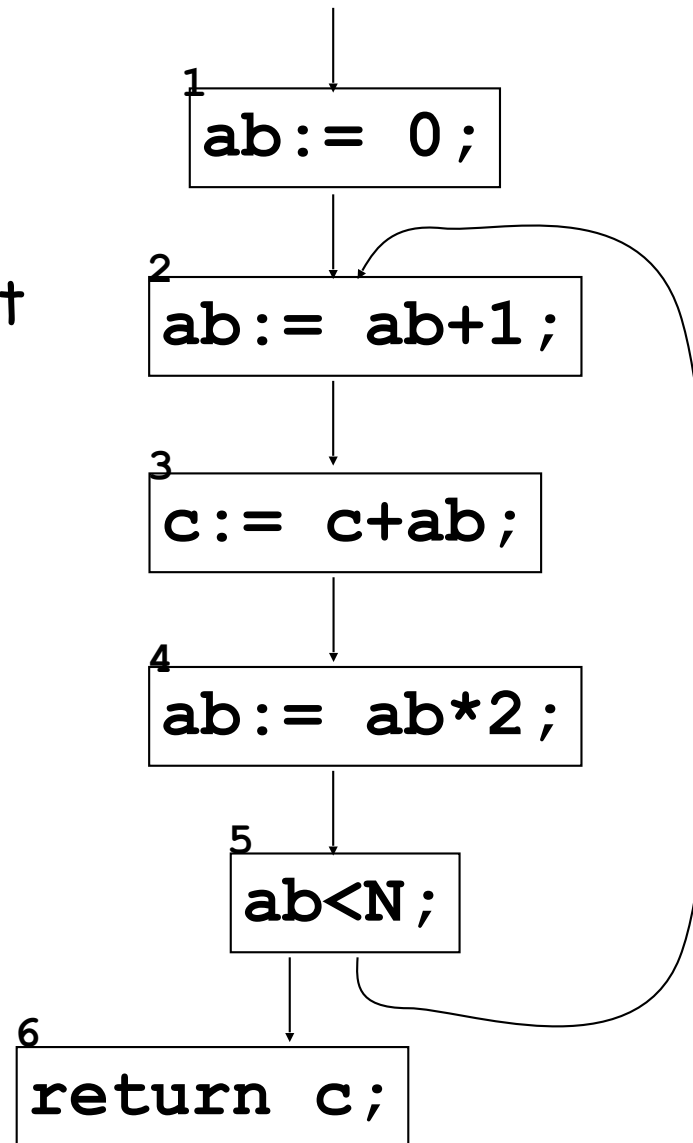
- **c** is live along all the arcs
- By the way: liveness analysis can be exploited to deduce that if **c** is a local variable then **c** will be used with no prior initialization (this information can be used by compilers to raise a warning message)





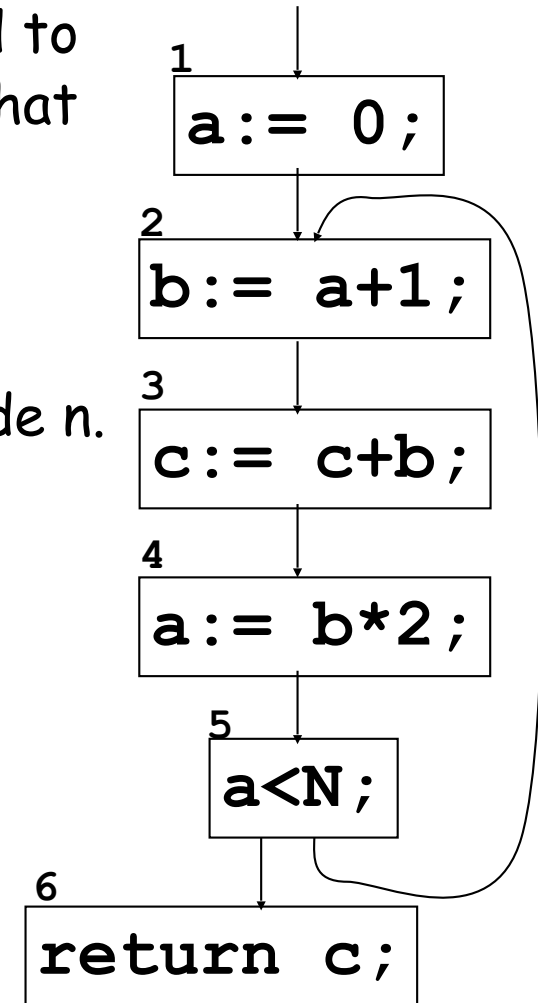
→ Two registers are enough: variables `a` and `b` will be never simultaneously live along the same arc

Variables **a** and **b** will be never simultaneously live along the same arc. Hence, instead of using two distinct variables **a** and **b** we can correctly employ a single variable **ab**



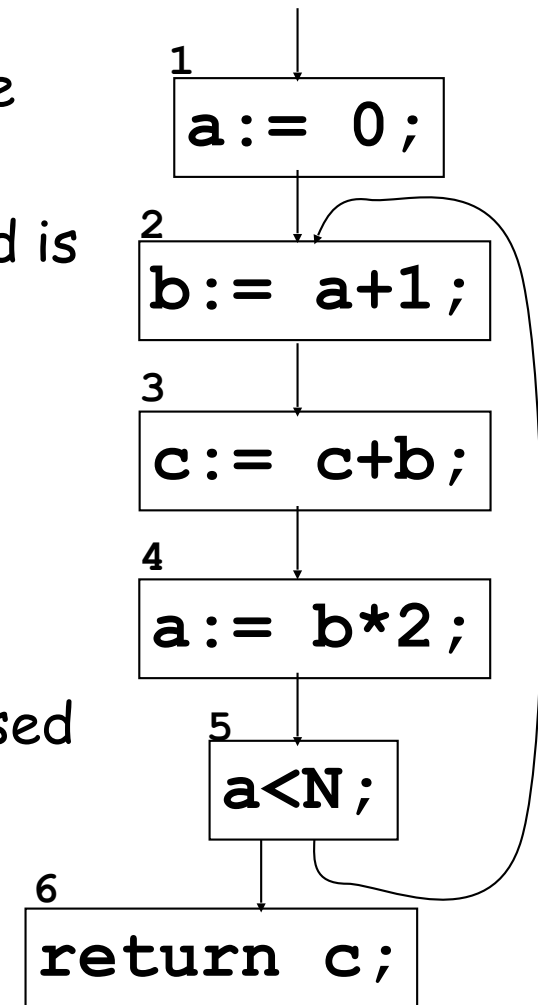
We need a way to compute live variables

- A CFG has outgoing edges (**out-edges**) that lead to successor nodes, and ingoing edges (**in-edges**) that originate from predecessor nodes.
- **pre[n]** and **post[n]** denote, respectively, the predecessors and successors nodes of some node n .
- As an example, in this CFG:
 - 2 and 6 are successors of node 5 because
 $5 \rightarrow 6$ and $5 \rightarrow 2$ are the out-edges of 5
 - 1 and 5 predecessor 2 since
 $5 \rightarrow 2$ and $1 \rightarrow 2$ are the in-edges of 2
 - $\text{pre}[2]=\{1,5\}$; $\text{post}[5]=\{2,6\}$.



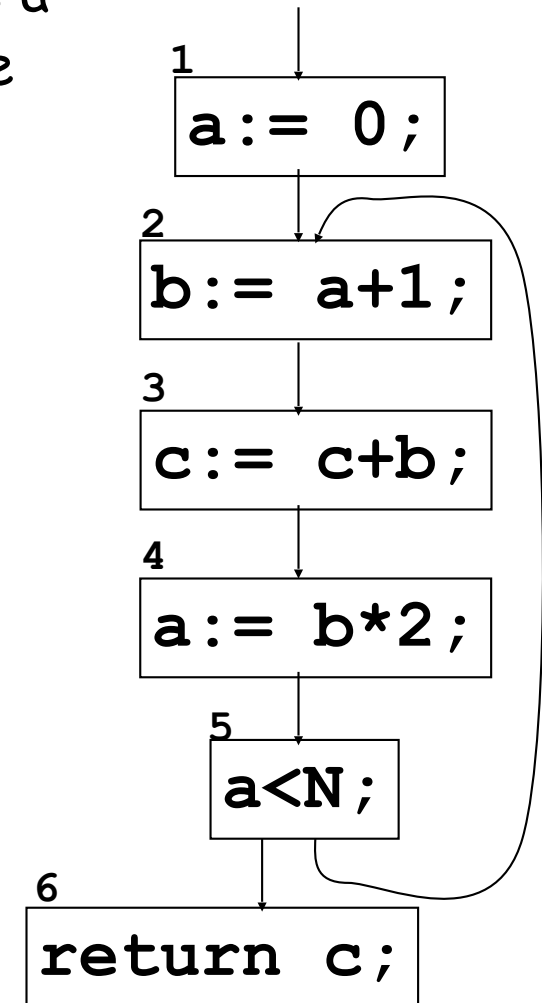
Notation

- An assignment to some variable (a use of the variable as L-value) is called a **definition** of the variable
- A use of some variable as R-value in a command is called a **use** of this variable
- **def[n]** denotes the set of variables that are defined in the node n
- **use[n]** denotes the set of variables that are used in the node n
- As an example, in this CFG:
 - $\text{def}[3] = \{c\}$, $\text{def}[5] = \emptyset$
 - $\text{use}[3] = \{b, c\}$, $\text{use}[5] = \{a\}$



Formalization of the property:

- A variable x is **live** along an arc $e \rightarrow f$ if there exists a **real execution path** P from the node e to some node n such that:
 - $e \rightarrow f$ is the first arc of such path P
 - $x \in \text{use}[n]$
 - for any node $n' \neq e$ and $n' \neq n$ in the path P , $x \notin \text{def}[n']$
- A variable x is **live-out** in some node n if x is live along **some** (i.e., at least one) out-edge of n
- A variable x is **live-in** in some node n if x is live along **any** in-edge of n



Example

As an example, in this CFG:

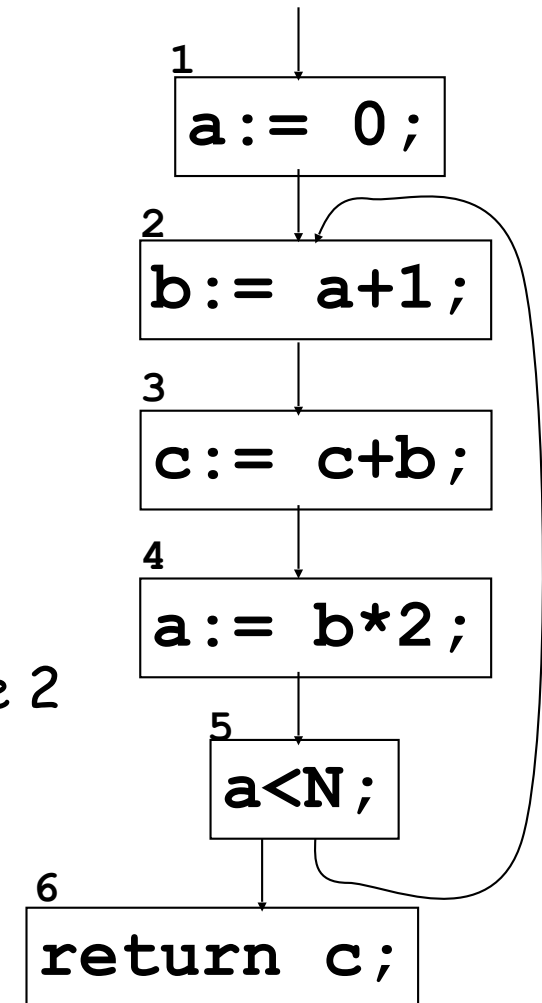
a is live along $1 \rightarrow 2$, $4 \rightarrow 5$ and $5 \rightarrow 2$

b is live along $2 \rightarrow 3$, $3 \rightarrow 4$

c is live along any arc

a is live-in in node 2, while it is not live-out in node 2

a is live-out in node 5



Computing an approximation of Liveness property

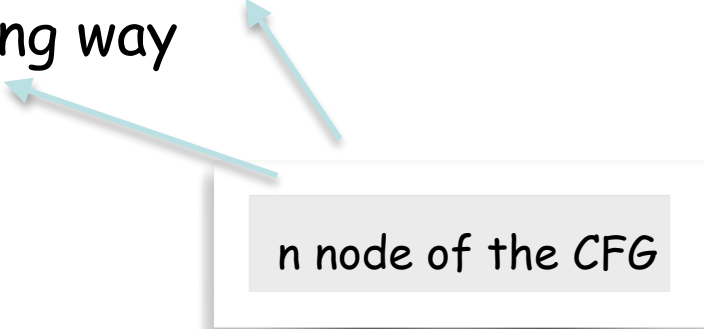
Let us define the following notation:

$in[n]$ is the set of variables that the static analysis determines to be live-in at node n

$out[n]$ is the set of variables that the static analysis determines to be live-out at node n

Computing an approximation of Liveness property

Liveness information: the sets $\text{in}[n]$ and $\text{out}[n]$ is computed as an over-approximation in the following way



n node of the CFG

1. If a variable $x \in \text{use}[n]$ then x is live-in in node n .
In other terms, if a node n uses a variable x as R-value then this variable x is live along each arc that enters into n .



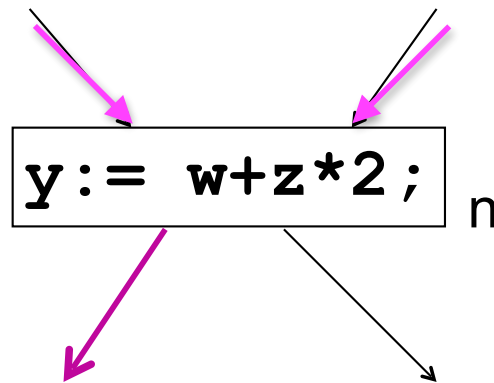
```
y := x+z+2;
n
```

$$\text{in}[n] \supseteq \text{use}[n]$$

Computing Liveness

2. If a variable x is live-out in a node n and $x \notin \text{def}[n]$ then the variable x is also live-in in this node n .

If a variable x is live for some arc that leaves a node n and x is not assigned in n then x is live for all the arcs that enter in n

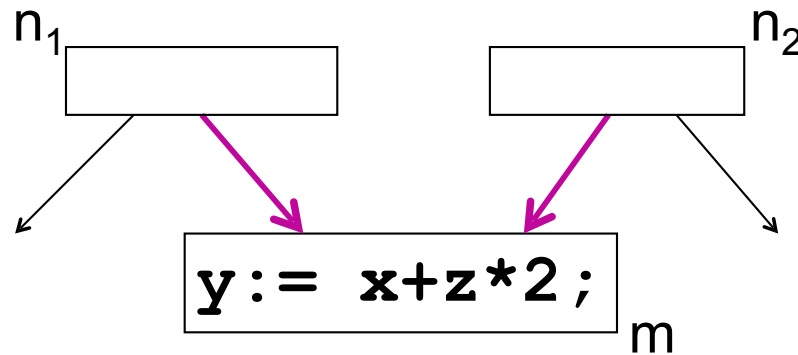


$$\text{in}[n] \supseteq \text{out}[n] - \text{def}[n]$$

Computing Liveness

3. If a variable x is live-in in a node m then x is live-out for all the nodes n such that $m \in \text{post}[n]$.

This is clearly correct by definition.



$$\text{out}[n_1] \supseteq \cup \{\text{in}[m] \mid m \in \text{post}[n_1]\}$$

$$\text{out}[n_2] \supseteq \cup \{\text{in}[m] \mid m \in \text{post}[n_2]\}$$

Dataflow Equations

The previous three rules of liveness analysis can be thus formalized by two equations for each node n :

1. $\text{in}[n] = \text{use}[n] \cup (\text{out}[n] - \text{def}[n])$ (rules 1 and 2)

2. $\text{out}[n] = \bigcup \{\text{in}[m] \mid m \in \text{post}[n]\}$ (rule 3)

Correctness of the analysis of Liveness

This definition of liveness analysis $\text{in}[n]$ and $\text{out}[n]$ is **correct**:

If x is concretely live-in (live-out) in some node n then the static analysis will detect that $x \in \text{in}[n]$ ($x \in \text{out}[n]$):

$$\begin{aligned}\text{in}[n] &\supseteq \text{live-in}[n] \\ \text{out}[n] &\supseteq \text{live-out}[n]\end{aligned}$$

In other terms, no actually live variable is neglected by liveness analysis.

Correctness in Dragon Book

Why the Available-Expressions Algorithm Works

We need to explain why starting all OUT's except that for the entry block with U , the set of all expressions, leads to a conservative solution to the data-flow equations; that is, all expressions found to be available really *are* available. First, because intersection is the meet operation in this data-flow schema, any reason that an expression $x + y$ is found not to be available at a point will propagate forward in the flow graph, along all possible paths, until $x + y$ is recomputed and becomes available again. Second, there are only two reasons $x + y$ could be unavailable:

1. $x + y$ is killed in block B because x or y is defined without a subsequent computation of $x + y$. In this case, the first time we apply the transfer function f_B , $x + y$ will be removed from $\text{OUT}[B]$.
2. $x + y$ is never computed along some path. Since $x + y$ is never in $\text{OUT}[\text{ENTRY}]$, and it is never generated along the path in question, we can show by induction on the length of the path that $x + y$ is eventually removed from IN's and OUT's along that path.

Thus, after changes subside, the solution provided by the iterative algorithm of Fig. 9.20 will include only truly available expressions.

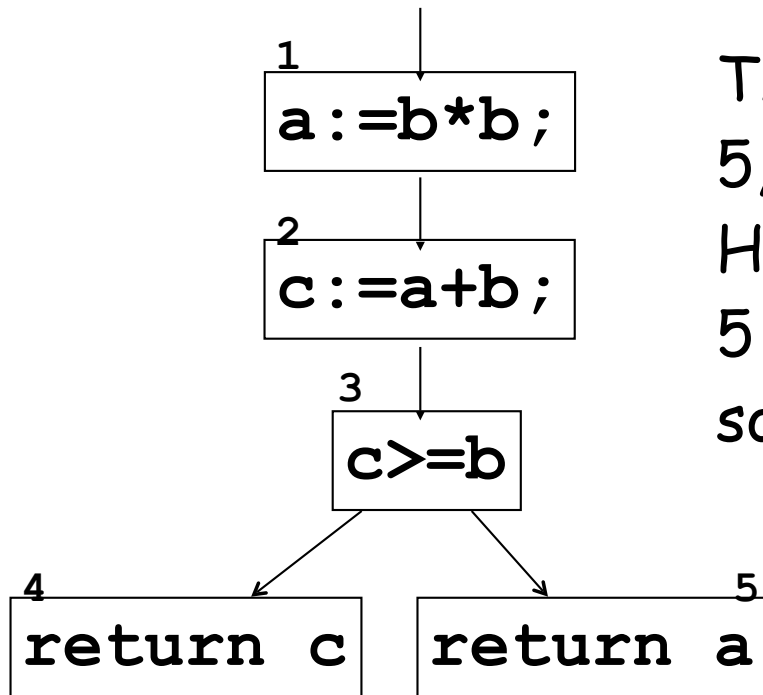
Computing Liveness

Liveness analysis is **approximate**:

it assumes that each path of the CFG is a **feasible path**
while this hypothesis is obviously not true

Computing Liveness

Liveness analysis is **approximate**: it assumes that each path of the CFG actually is a **feasible path** while this hypothesis is obviously not true.



The analysis determines that a is live-in in 5, and therefore a is live-out in 3. However, no real execution path from 3 to 5 exists (because $b + b * b < b$ is always false) so that a is not really live when exiting 3!

How can we compute a solution to 1 and 2?

1. $\text{in}[n] = \text{use}[n] \cup (\text{out}[n] - \text{def}[n])$

2. $\text{out}[n] = \bigcup \{ \text{in}[m] \mid m \in \text{post}[n] \}$

Correctness tells us that $\text{in}[n] \supseteq \text{live-in}[n]$ and $\text{out}[n] \supseteq \text{live-out}[n]$

But we need a way to compute Live variable analysis

How can we compute a solution to 1 and 2?

1. $\text{in}[n] = \text{use}[n] \cup (\text{out}[n] - \text{def}[n])$
2. $\text{out}[n] = \bigcup \{ \text{in}[m] \mid m \in \text{post}[n] \}$

We need to compute for each node the sets that satisfy the previous equations: fix points

- but how can we be sure that such fix-points exist?

It depends on the **domain** and on the **function**!

What do we need?

- to settle a domain: $\text{CPO}_\perp (D, \sqsubseteq)$
- a function $f : D \rightarrow D$ continuous

then we can apply Kleene 's Theorem

On finite sets

If a poset $(D, \sqsubseteq)$ is finite then it is a **CPO**

If a poset $(D, \sqsubseteq)$ is finite and $f : D \rightarrow D$ is monotone
then $f : D \rightarrow D$ is continuous

1. $in[n] = use[n] \cup (out[n] - def[n])$
2. $out[n] = \bigcup \{in[m] \mid m \in post[n]\}$

Which is our domain?

Our objects:

Given a node we need to compute the set **in** and the set **out** (sets of variables)

- Let **Vars** be the finite set of variables that occur in the program *P* to analyze. All possible subsets: $\mathcal{P}(\text{Vars})$

For each node we need a set **in** and a set **out**: $\mathcal{P}(\text{Vars}) \times \mathcal{P}(\text{Vars})$

Our CFG has *N* nodes, so our **domain** will be

$(\mathcal{P}(\text{Vars}) \times \mathcal{P}(\text{Vars}))^N$: *N*-tuples of pairs of subsets of **Vars**

The **order** : $\subseteq^{2N}$

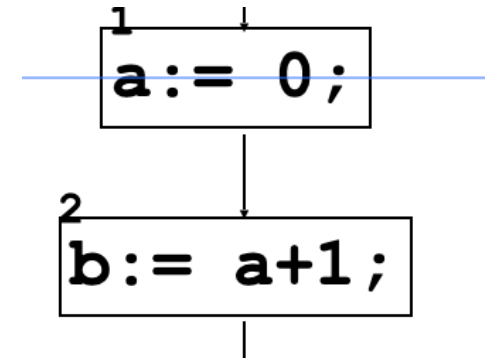
$$\langle in_1^1, out_1^1, \dots, in_N^1, out_N^1 \rangle \subseteq^{2N} \langle in_1^2, out_1^2, \dots, in_N^2, out_N^2 \rangle \text{ iff}$$

$$\forall i, \quad in_i^1 \subseteq in_i^2 \text{ and } out_i^1 \subseteq out_i^2$$

Example

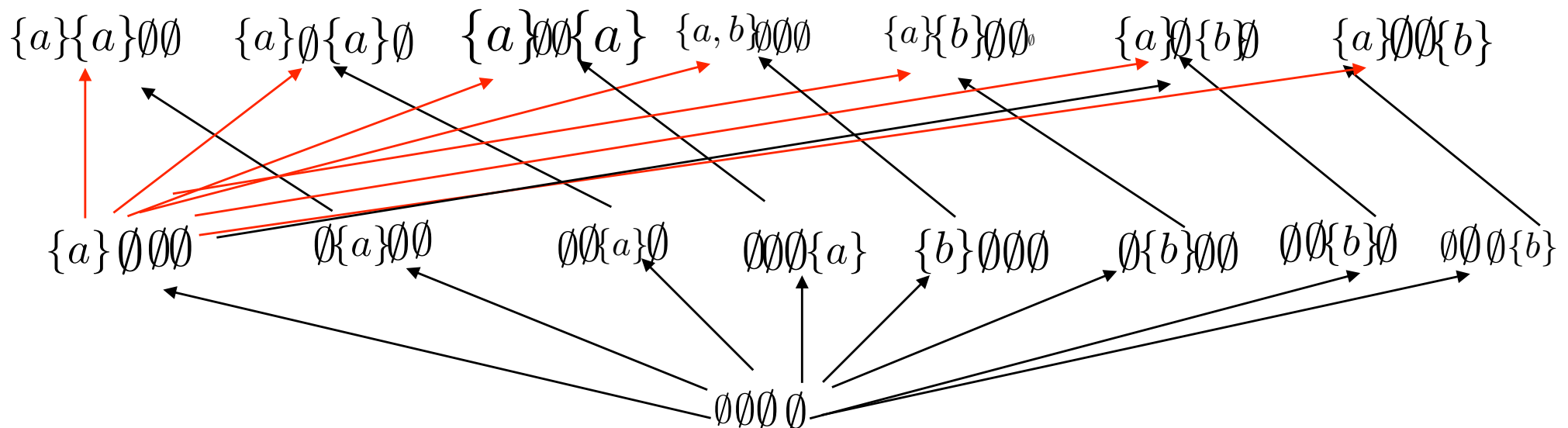
Vars = {a,b} N=2.

$\langle \mathcal{P}(\text{Vars}) \times \mathcal{P}(\text{Vars})^2, \subseteq^4 \rangle$ is a finite domain.



$\{a, b\} \{a, b\} \{a, b\} \{a, b\}$

...



Our domain

$$\langle (\mathcal{P}(\text{Vars}) \times \mathcal{P}(\text{Vars}))^{\mathbb{N}}, \subseteq^{2N} \rangle$$

CPO with bottom?

It is a CPO because it is finite
bottom?

1. $in[n] = use[n] \cup (out[n] - def[n])$
2. $out[n] = \bigcup \{in[m] \mid m \in post[n]\}$

Which is our function?

The map Live:

$(\mathcal{P}(\text{Vars}) \times \mathcal{P}(\text{Vars}))^N \rightarrow (\mathcal{P}(\text{Vars}) \times \mathcal{P}(\text{Vars}))^N$ defined by

$\text{Live}(\langle in_1, out_1, \dots, in_N, out_N \rangle) =$

$\langle use[1] \cup (out_1 - def[1]), \bigcup_{m \in post[1]} in_m, \dots, use[N] \cup (out_N - def[N]), \bigcup_{m \in post[N]} in_m \rangle$

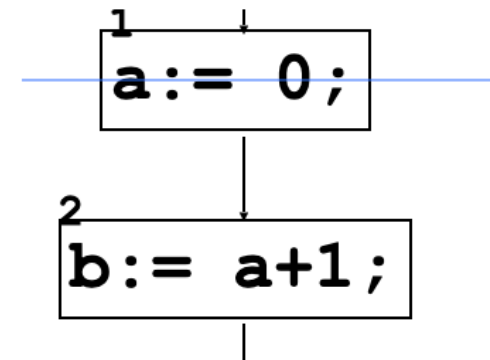
• Example

$\text{Live}(\langle \{\}\{\}\{\}\{\}\rangle) = \langle \{\}\{\}\{a\}\{\}\rangle$

$\text{Live}(\langle \{\}\{\}\{a\}\{\}\rangle) = \langle \{\}\{a\}\{a\}\{\}\rangle$

$\text{Live}(\langle \{\}\{a\}\{a\}\{\}\rangle) = \langle \{\}\{a\}\{a\}\{\}\rangle$

Note that Live is monotone!



$def(1) = \{a\}, use(1) = \{\}$
 $def(2) = \{b\}, use(2) = \{a\}$

Is it continuous?

The map Live:

$((\mathbf{Vars}) \times \mathcal{P}(\mathbf{Vars}))^N \rightarrow (\mathcal{P}(\mathbf{Vars}) \times \mathcal{P}(\mathbf{Vars}))^N$ defined by

$\text{Live}(\langle in_1, out_1, \dots, in_N, out_N \rangle) =$

$\langle use[1] \cup (out_1 - def[1]), \bigcup_{m \in post[1]} in_m, \dots, use[N] \cup (out_N - def[N]), \bigcup_{m \in post[N]} in_m \rangle$

is continuous?

Yes! because it is **monotone on a finite domain**